BTEUP EXAM NOTES | ENGLISH EDITION

Mobile App Development

Unit III: User Interface Design Essentials: Widgets, Layouts, Animation

About these notes

Part A explains every topic in very simple English with real-life examples, so a beginner can understand it from zero.
Part B gives important exam questions with full 5-mark model answers, ready to write in the exam.
At the end: quick revision sheet, common mistakes, and a self-test.

Syllabus covered (Unit III)

Android User Interface Design Essentials: User Interface Screen elements, Designing User Interfaces with Layouts, Drawing and Working with Animation.

Unit Overview and Contents

Simple idea: The user interface (UI) is **everything the user sees and touches**: text, buttons, images, lists. This unit has three parts: (1) the **UI elements** (views and widgets), (2) **layouts** that arrange them on the screen, and (3) **drawing and animation** to make the screen graphic and lively.

Important note: Views vs Jetpack Compose

Your syllabus teaches the classic **View + XML layout** system, so these notes follow it. But in **May 2026 Google announced that Android UI development is 'Compose first'**: it recommends **Jetpack Compose** for all new features. The View system keeps working (critical bug fixes only, no new features) and a lot of existing apps still use it. Learn Views for the exam; a small Compose comparison table is given at the end of the layouts topic.

Topic	Exam importance	Where in notes
UI basics: View, ViewGroup, attributes	High	Part A: Topic 1 | Part B: Q1, Q14
UI elements (widgets)	Very High	Part A: Topic 2 | Part B: Q2, Q3, Q4, Q5
Layouts	Very High (almost sure)	Part A: Topic 3 | Part B: Q6, Q7, Q8, Q9
Drawing: text, bitmaps, shapes, canvas	Very High	Part A: Topic 4 | Part B: Q10
Animation: tween, frame, property	Very High	Part A: Topic 5 | Part B: Q11, Q12, Q13

Contents

Part A: Easy Explanation	Part B: Important Questions with Answers
Topic 1: UI basics Topic 2: UI screen elements Topic 3: Designing UI with layouts Topic 4: Drawing (text, bitmaps, shapes, canvas) Topic 5: Working with animation	Q1 View, ViewGroup, UI hierarchy (Likely) Q2 TextView and EditText (Very Likely) Q3 Button, CheckBox, RadioButton, Switch (Very Likely) Q4 Spinner and ListView (Very Likely) Q5 ProgressBar, SeekBar, RatingBar, pickers (Likely) Q6 Layouts and their types (Very Likely) Q7 LinearLayout (Very Likely) Q8 RelativeLayout (Very Likely) Q9 Frame, Table, Grid, Constraint layouts (Likely) Q10 Drawing: Canvas, Paint, shapes, bitmaps (Very Likely) Q11 Types of animation (Very Likely) Q12 Tween animation (Very Likely) Q13 Frame animation (Likely) Q14 Differences (Possible)

How to study this unit (suggested time: 1 day)

1) Read Part A once to understand. 2) Learn the **XML of each layout** and the **four tween animations** (alpha, scale, translate, rotate): they are asked as programs. 3) Practice one widget program (button click, spinner) and one animation program. 4) Revise with the Quick Revision Sheet and attempt the Self-Test.

PART A: Easy Explanation (Simple English)

Topic 1: UI Basics: View and ViewGroup

Real-life example

Think of a **photo frame wall**. The wall is the screen. Each photo frame is a **View** (a button, a text, an image). A **big board** that holds several frames in a neat order is a **ViewGroup** (a layout).

Definition

A **View** is the basic building block of the user interface; it draws something on the screen and handles user events (TextView, Button). A **ViewGroup** is an invisible container that holds other views and arranges them (LinearLayout, RelativeLayout).

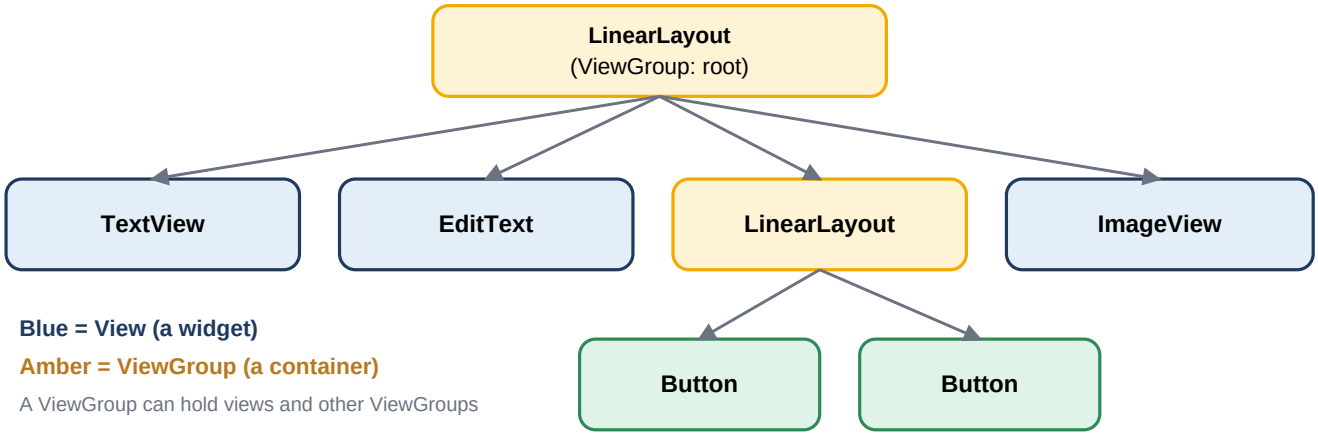


Figure 1: View hierarchy: ViewGroups contain Views

Ways to create a UI

- **XML layout files** (res/layout): the recommended way for Views. The design is separated from the code.
- **Layout Editor** in Android Studio: drag and drop views; it writes the XML for you.
- **Java/Kotlin code:** create views with `new TextView(this)` and add them to a layout. Good for dynamic UIs.
- **Jetpack Compose** (newer): write the UI as Kotlin functions instead of XML.

Common view attributes

Attribute	Meaning
android:id	Unique name: `@+id/btnOk`. Used with findViewById().
layout_width / layout_height	**match_parent** (fill the parent), **wrap_content** (just big enough), or a size in **dp**
android:text	The text shown (use @string/name)
textSize, textColor, textStyle	Size (in **sp**), colour, bold/italic
hint	Light help text in an empty EditText
padding / layout_margin	Space inside the view / space outside the view
background	Colour or drawable behind the view
gravity / layout_gravity	Position of the content inside the view / position of the view inside its parent

Attribute	Meaning
visibility	visible, invisible (hidden but takes space), gone (hidden, no space)
onClick	Name of the method to call when clicked

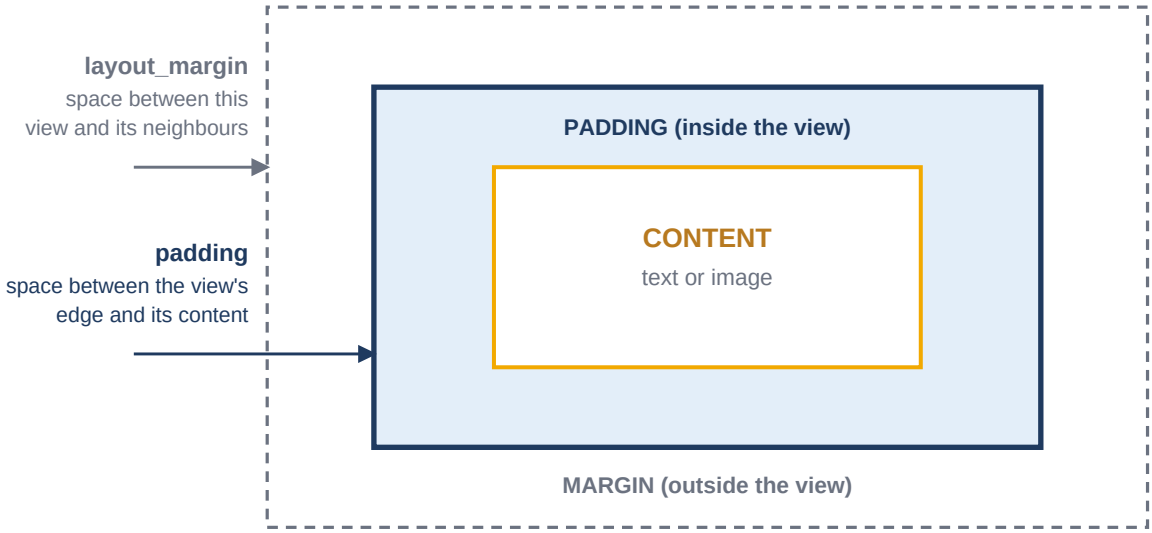


Figure 2: Margin, padding and content of a view

Unit	Full form and use
dp (or dip)	Density-independent pixel. Use for sizes and margins; looks the same size on all screens.
sp	Scale-independent pixel. Use for **text size**; follows the user's font-size setting.
px	Pixel. Avoid: its size changes with screen density.

Handling events (making a view respond)

- **findViewById(R.id.name)** connects the view in XML to a variable in code.
- **setOnClickListener()** sets what happens when the view is clicked. Other listeners: **OnLongClickListener**, **OnCheckedChangeListener**, **OnItemSelectedListener**, **OnSeekBarChangeListener**.
- The older shortcut is `android:onClick="submit"` in XML plus a method `public void submit(View v)` in the activity.

Topic 2: User Interface Screen Elements (Widgets)

Android gives ready-made UI elements called **widgets**. They are grouped by the job they do.

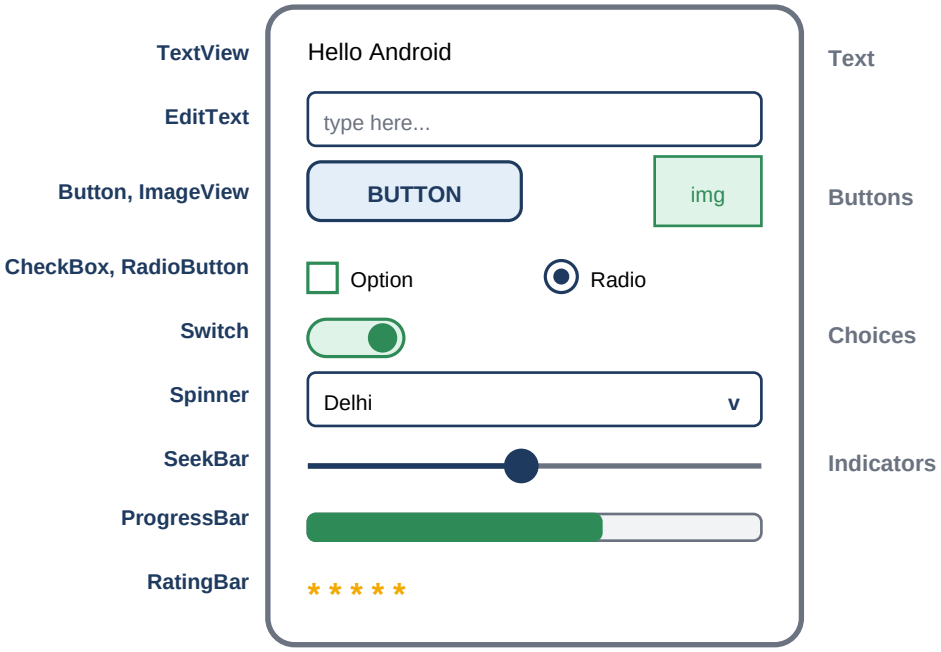


Figure 3: Common widgets on a screen

Group	Widgets
Text	TextView, EditText, AutoCompleteTextView
Buttons	Button, ImageButton, CheckBox, RadioButton / RadioGroup, ToggleButton, Switch
Indicators	ProgressBar, SeekBar, RatingBar
Pickers and clocks	DatePicker, TimePicker, TextClock, Chronometer
Adapter views	Spinner, ListView, GridView (RecyclerView is the modern list)
Other	ImageView, Toast, Snackbar, AlertDialog

2(a) Text widgets: TextView and EditText

- **TextView:** shows text; the user cannot edit it. Attributes: text, textSize, textColor, textStyle, fontFamily, gravity, maxLines.
- **EditText:** a text box where the user types. Important attribute **inputType**: text, textPassword, number, phone, textEmailAddress, textMultiLine. Read the value with `getText().toString()`.
- **AutoCompleteTextView:** an EditText that shows suggestions while typing (needs an ArrayAdapter).

activity_main.xml: TextView, EditText and Button

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Login"
        android:textSize="24sp"
        android:textStyle="bold" />

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter your name"
        android:inputType="textPersonName" />

    <Button
        android:id="@+id/btnSubmit"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Submit" />
</LinearLayout>
```

MainActivity.java (inside onCreate): handle the button click

```
EditText etName = findViewById(R.id.etName);
Button btnSubmit = findViewById(R.id.btnSubmit);

btnSubmit.setOnClickListener(v -> {
    String name = etName.getText().toString();
    Toast.makeText(this, "Hello " + name, Toast.LENGTH_SHORT).show();
});
```

2(b) Buttons: Button, CheckBox, RadioButton, ToggleButton, Switch

Widget	Use	Listener / method
Button	Performs an action when tapped	setOnClickListener()
ImageButton	A button with an image instead of text	setOnClickListener()
CheckBox	Two-state box; many can be selected together	setOnCheckedChangeListener(), isChecked()
RadioButton in a **RadioGroup**	Only **one** choice of the group can be selected	RadioGroup.setOnCheckedChangeListener()
ToggleButton / **Switch**	ON / OFF control	setOnCheckedChangeListener(), isChecked()

XML: CheckBox and RadioGroup

```
<CheckBox android:id="@+id/cbAgree" android:layout_width="wrap_content"
    android:layout_height="wrap_content" android:text="I agree" />

<RadioGroup android:id="@+id/rgGender" android:layout_width="wrap_content"
    android:layout_height="wrap_content" android:orientation="horizontal">
    <RadioButton android:id="@+id/rbMale" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Male" />
    <RadioButton android:id="@+id/rbFemale" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Female" />
</RadioGroup>
```

Java: handle CheckBox and RadioGroup

```
CheckBox cb = findViewById(R.id.cbAgree);
cb.setOnCheckedChangeListener((buttonView, isChecked) ->
        Toast.makeText(this, isChecked ? "Agreed" : "Not agreed",
                       Toast.LENGTH_SHORT).show());

RadioGroup rg = findViewById(R.id.rgGender);
rg.setOnCheckedChangeListener((group, checkedId) -> {
    RadioButton rb = findViewById(checkedId);
    Toast.makeText(this, rb.getText(), Toast.LENGTH_SHORT).show();
});
```

2(c) Adapter views: Spinner and ListView

Real-life example

An adapter is a **waiter**: it takes the data (the kitchen's list of dishes) and serves each item on the screen in a ready-made row.

- An **AdapterView** shows a list of items taken from a data source through an **Adapter** (ArrayAdapter, SimpleAdapter, CursorAdapter).
- **Spinner:** a drop-down list; the user picks one item. **ListView:** a scrolling vertical list. **GridView:** items in a scrolling grid.
- **RecyclerView** is the modern, faster list view and is preferred for new apps. ListView is still supported and is simpler to learn. **Gallery** was deprecated in API 16.

Spinner data and XML

```
<!-- res/values/strings.xml -->
<string-array name="cities">
    <item>Delhi</item>
    <item>Mumbai</item>
    <item>Chennai</item>
    <item>Kolkata</item>
</string-array>

<!-- layout -->
<Spinner android:id="@+id/spCity" android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

Java: Spinner with ArrayAdapter

```
Spinner sp = findViewById(R.id.spCity);
ArrayAdapter<CharSequence> adapter = ArrayAdapter.createFromResource(
        this, R.array.cities, android.R.layout.simple_spinner_item);
adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
sp.setAdapter(adapter);

sp.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener() {
    @Override
    public void onItemSelected(AdapterView<?> parent, View view, int pos, long id) {
        String city = parent.getItemAtPosition(pos).toString();
        Toast.makeText(MainActivity.this, city, Toast.LENGTH_SHORT).show();
    }
    @Override
    public void onNothingSelected(AdapterView<?> parent) { }
});
```

Java: ListView with ArrayAdapter

```
String[] items = {"Android", "Java", "Kotlin", "XML"};
ListView lv = findViewById(R.id.listView);          // <ListView android:id="@+id/listView" .../>
ArrayAdapter<String> ad = new ArrayAdapter<>(
        this, android.R.layout.simple_list_item_1, items);
lv.setAdapter(ad);
lv.setOnItemClickListener((parent, view, position, id) ->
        Toast.makeText(this, items[position], Toast.LENGTH_SHORT).show());
```

2(d) Indicators, pickers and other elements

Widget	Purpose	Key points
ProgressBar	Shows progress or 'loading'	Circular (indeterminate) or horizontal with max and progress; setProgress()
SeekBar	Slider the user drags	OnSeekBarChangeListener: onProgressChanged()
RatingBar	Star rating	numStars, stepSize, getRating()
DatePicker / TimePicker	Choose a date / time	DatePickerDialog and TimePickerDialog show them as dialogs. **Month starts at 0** (January = 0)
TextClock / Chronometer	Show current time / a stopwatch	**AnalogClock** (deprecated API 23) and **DigitalClock** (deprecated API 17) are old; use TextClock
ImageView	Shows an image	src, scaleType (centerCrop, fitCenter), contentDescription
Toast	Small message for a moment	Toast.makeText(ctx, text, LENGTH_SHORT).show()
Snackbar	Message at the bottom, can have an action	Material library: Snackbar.make(view, text, LENGTH_LONG).show()
AlertDialog	Pop-up asking for a decision	AlertDialog.Builder. **ProgressDialog** is deprecated (API 26)

XML: horizontal ProgressBar

```
<ProgressBar android:id="@+id/progress" style="?android:attr/progressBarStyleHorizontal"
    android:layout_width="match_parent" android:layout_height="wrap_content"
    android:max="100" android:progress="40" />
```

Java: SeekBar, DatePickerDialog and AlertDialog

```
SeekBar seek = findViewById(R.id.seekBar);
seek.setOnSeekBarChangeListener(new SeekBar.OnSeekBarChangeListener() {
    @Override public void onProgressChanged(SeekBar bar, int progress, boolean fromUser) {
        textView.setText("Value: " + progress);
    }
    @Override public void onStartTrackingTouch(SeekBar bar) { }
    @Override public void onStopTrackingTouch(SeekBar bar) { }
});

// Date picker dialog (month 0 = January)
new DatePickerDialog(this, (view, year, month, day) ->
        textView.setText(day + "/" + (month + 1) + "/" + year), 2026, 0, 1).show();

// Alert dialog
new AlertDialog.Builder(this)
        .setTitle("Exit")
        .setMessage("Do you want to exit?")
        .setPositiveButton("Yes", (dialog, which) -> finish())
        .setNegativeButton("No", null)
        .show();
```

Topic 3: Designing User Interfaces with Layouts

Real-life example

A layout is the **seating plan of a hall**. It decides where each guest (view) sits: in one line, relative to another guest, stacked, or in a table. Different halls need different plans.

Definition

A **layout** is a **ViewGroup** that defines the structure and arrangement of the views on a screen. Layouts are written in **XML** (res/layout) or created in code.

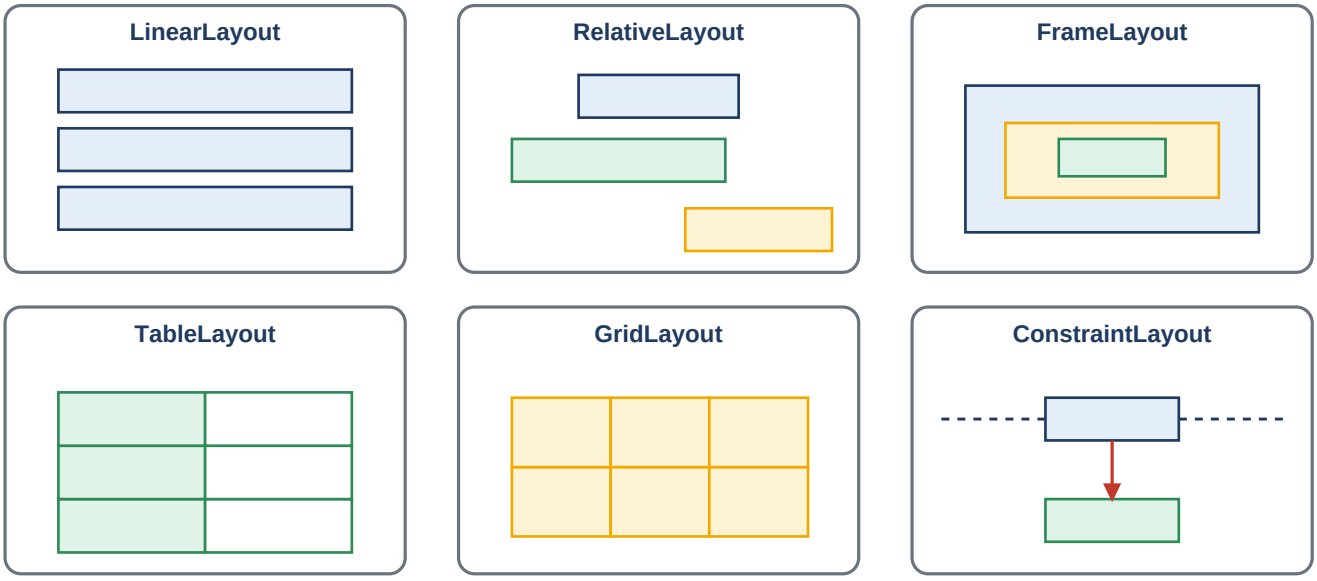


Figure 4: Main layout types

Layout	How it arranges children	Best for
LinearLayout	In a single row or column (orientation)	Simple forms, buttons in a row
RelativeLayout	Each view relative to the parent or to other views	Screens where positions depend on each other
FrameLayout	Views stacked on top of each other	One view or overlays (text on an image)
TableLayout	Rows (TableRow) and columns	Tabular data, label + field forms
GridLayout	A grid of rows and columns with spanning	Calculators, keypads
ConstraintLayout	Each view has constraints to the parent or other views; flat hierarchy	Complex, responsive screens (default in Android Studio templates)

- **AbsoluteLayout** (fixed x, y positions) is deprecated since API level 3; do not use it.
- **Layouts can be nested**, but deep nesting makes the UI slow. Prefer ConstraintLayout for complex screens to keep the hierarchy flat.
- Wrap long screens in a **ScrollView** so the user can scroll (it can have only one direct child).

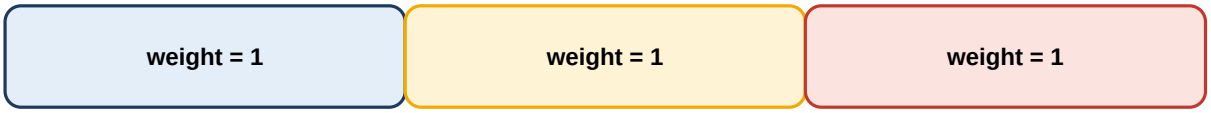
3(a) LinearLayout

- Arranges children in one direction: **android:orientation** = **vertical** (top to bottom) or **horizontal** (left to right). The default is horizontal.
- **layout_weight** shares the free space among children in a ratio. Set the width (horizontal) or height (vertical) to **0dp** when using weight.
- **gravity** positions the content inside a view; **layout_gravity** positions the view inside its parent.

layout_weight 1 and 2 (width = 0dp)



three views with equal weight 1 : 1 : 1



Free space is shared in the ratio of the weights.

Figure 5: layout_weight in a horizontal LinearLayout

LinearLayout with weights 1 : 2 : 1

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal">

    <Button android:layout_width="0dp" android:layout_height="wrap_content"
        android:layout_weight="1" android:text="One" />
    <Button android:layout_width="0dp" android:layout_height="wrap_content"
        android:layout_weight="2" android:text="Two" />
    <Button android:layout_width="0dp" android:layout_height="wrap_content"
        android:layout_weight="1" android:text="Three" />
</LinearLayout>
```

3(b) RelativeLayout

- Each child is positioned **relative to the parent** (layout_alignParentTop, layout_alignParentEnd, layout_centerInParent) or **relative to a sibling view** by its id (layout_below, layout_above, layout_toRightOf / layout_toEndOf, layout_alignLeft).
- Gives flexible designs without deep nesting, but views must be declared in a good order and ids referenced as `@id/name`.

RelativeLayout: a label, a text box and a button

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">

    <TextView
        android:id="@+id/tvName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerHorizontal="true"
        android:text="Name" />

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_below="@id/tvName" />

    <Button
        android:id="@+id/btnOk"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@id/etName"
        android:layout_alignParentEnd="true"
        android:text="OK" />
</RelativeLayout>
```

3(c) FrameLayout, TableLayout, GridLayout

FrameLayout, TableLayout and GridLayout

```
<!-- FrameLayout: text placed over an image -->
<FrameLayout android:layout_width="match_parent" android:layout_height="200dp">
    <ImageView android:layout_width="match_parent" android:layout_height="match_parent"
        android:src="@drawable/photo" android:scaleType="centerCrop"
        android:contentDescription="Photo" />
    <TextView android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:layout_gravity="bottom|center_horizontal"
        android:text="Caption" android:textColor="#FFFFFF" />
</FrameLayout>

<!-- TableLayout: label + field rows -->
<TableLayout android:layout_width="match_parent" android:layout_height="wrap_content"
    android:stretchColumns="1">
    <TableRow>
        <TextView android:text="Name:" />
        <EditText android:hint="Enter name" />
    </TableRow>
    <TableRow>
        <TextView android:text="Email:" />
        <EditText android:hint="Enter email" />
    </TableRow>
</TableLayout>

<!-- GridLayout: 2 columns -->
<GridLayout android:layout_width="match_parent" android:layout_height="wrap_content"
    android:columnCount="2">
    <Button android:text="1" /> <Button android:text="2" />
    <Button android:text="3" /> <Button android:text="4" />
</GridLayout>
```

- **FrameLayout:** the last child is drawn on top. Use **layout_gravity** to place each child.
- **TableLayout:** each **TableRow** is a row; the number of columns is the largest row. **stretchColumns** makes a column fill the extra width.
- **GridLayout: columnCount** and **rowCount** define the grid; children can span cells.

3(d) ConstraintLayout

- Each view needs **at least one horizontal and one vertical constraint**, for example `layout_constraintTop_toTopOf="parent"`.
- **0dp** here means 'match constraints' (fill the space between the constraints).
- Gives a **flat hierarchy**, so complex screens are fast; the Layout Editor can create constraints by dragging.

ConstraintLayout

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent" android:layout_height="match_parent">

    <TextView android:id="@+id/tvTitle"
        android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:text="Title"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <Button android:id="@+id/btnGo"
        android:layout_width="0dp" android:layout_height="wrap_content"
        android:text="Go"
        app:layout_constraintTop_toBottomOf="@id/tvTitle"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

UI created in Java code

```
// Creating a UI in code instead of XML
LinearLayout ll = new LinearLayout(this);
ll.setOrientation(LinearLayout.VERTICAL);

TextView tv = new TextView(this);
tv.setText("Hello from code");
Button btn = new Button(this);
btn.setText("Click");

ll.addView(tv);
ll.addView(btn);
setContentView(ll);
```

View system (XML)	Jetpack Compose (Kotlin)
LinearLayout vertical / horizontal	Column / Row
FrameLayout	Box
ConstraintLayout	ConstraintLayout (Compose version)
TextView / EditText / Button / ImageView	Text / TextField / Button / Image
RecyclerView / ListView	LazyColumn / LazyRow
Design is in XML files; code uses findViewById	Design is written in Kotlin functions marked @Composable

Topic 4: Drawing: Text, Bitmaps and Shapes

Real-life example

Drawing in Android is like **painting on a canvas**. You need a **canvas** (the surface), a **brush and colours** (Paint), and something to paint (shapes, text, pictures).

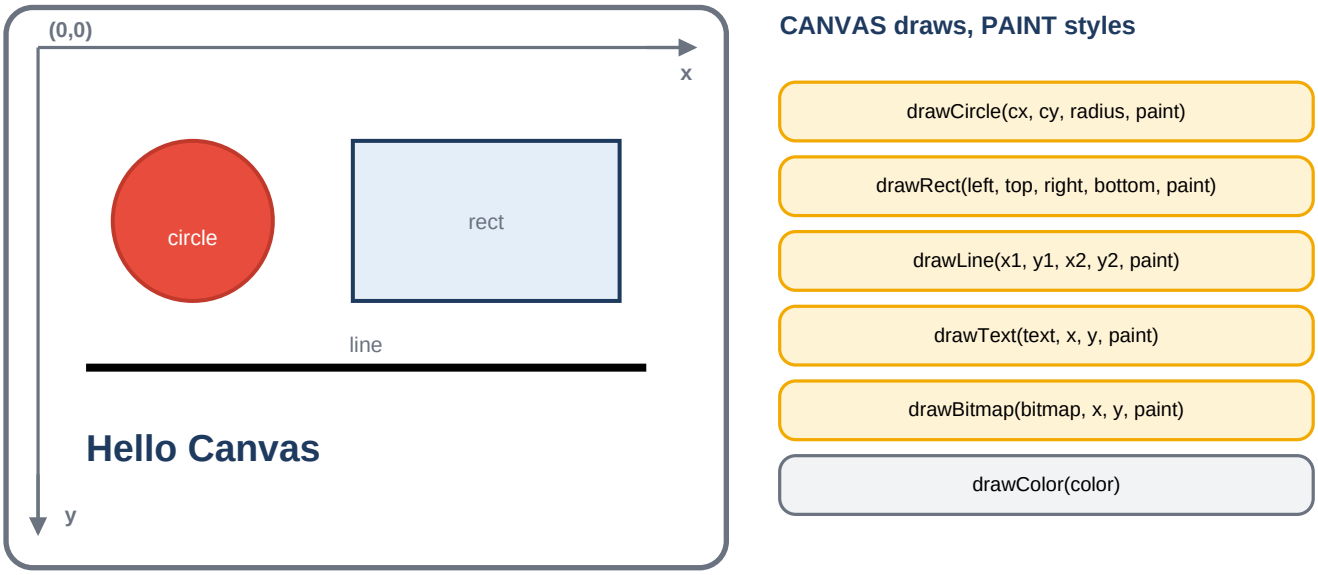


Figure 6: Canvas coordinates and drawing methods

Class	Role
Canvas	Provides the drawing methods: drawCircle(), drawRect(), drawLine(), drawText(), drawBitmap(), drawColor()
Paint	Holds how to draw: colour, style (FILL, STROKE), stroke width, text size, anti-aliasing
Bitmap	The pixel image that is drawn on or drawn
Path	A custom outline made of lines and curves
Drawable	Anything that can be drawn: bitmap, shape, vector, animation. Used as a src or background

- The coordinate origin **(0, 0) is the top-left corner**; x grows to the right and y grows **downwards**. Units are pixels.

4(a) Working with text

- **In layouts:** style a TextView with **textSize** (sp), **textColor**, **textStyle** (bold, italic) and **fontFamily** (for example sans-serif-medium or a custom font in res/font).
- **In code:** `canvas.drawText("Hello", x, y, paint)` with `paint.setTextSize()`. A custom font can be loaded with `Typeface.createFromAsset()` or from **res/font**.

4(b) Working with bitmaps

XML: show an image

```
<ImageView android:layout_width="200dp" android:layout_height="200dp"
    android:src="@drawable/photo" android:scaleType="centerCrop"
    android:contentDescription="My photo" />
```

Java: load, resize and draw a bitmap

```
Bitmap bmp = BitmapFactory.decodeResource(getResources(), R.drawable.photo);
imageView.setImageBitmap(bmp);                              // show it
Bitmap small = Bitmap.createScaledBitmap(bmp, 200, 200, true); // resize
canvas.drawBitmap(bmp, 50, 50, null);                       // draw on a canvas (in onDraw)
```

- Big bitmaps use a lot of memory and can cause **OutOfMemoryError**. Load a smaller version with **inSampleSize** or use an image library (**Glide, Coil, Picasso**).
- **scaleType** options: fitCenter, centerCrop, centerInside, fitXY.

4(c) Working with shapes

XML shape drawable (res/drawable): the easy way to make rounded buttons, circles and gradients without images. The **shape** can be **rectangle, oval, line or ring**.

A shape drawable and how to use it

```
<?xml version="1.0" encoding="utf-8"?>
<!-- res/drawable/rounded_box.xml -->
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="#1F3A5F" />
    <corners android:radius="12dp" />
    <stroke android:width="2dp" android:color="#F2A900" />
    <padding android:left="16dp" android:top="8dp" android:right="16dp" android:bottom="8dp" />
</shape>

<!-- gradient option, instead of <solid> -->
<!-- <gradient android:startColor="#1F3A5F" android:endColor="#0E7C86" android:angle="45" /> -->

<!-- use it -->
<TextView android:layout_width="wrap_content" android:layout_height="wrap_content"
    android:background="@drawable/rounded_box" android:text="Hello" android:textColor="#FFFFFF" />
```

- **Tags inside a shape:** solid (fill colour), stroke (border), corners (rounded), gradient, padding, size.
- **Tip:** on new Material themes a plain Button may ignore `android:background`. Use `backgroundTint` or apply the shape to a TextView / layout.
- **In code:** `ShapeDrawable sd = new ShapeDrawable(new OvalShape()); sd.getPaint().setColor(Color.RED); view.setBackground(sd);`

Custom drawing with a custom View (onDraw)

DrawView.java: drawing a circle, rectangle, line and text

```
public class DrawView extends View {
    private final Paint paint = new Paint(Paint.ANTI_ALIAS_FLAG);

    public DrawView(Context context) { super(context); }
    public DrawView(Context context, AttributeSet attrs) { super(context, attrs); }

    @Override
    protected void onDraw(Canvas canvas) {
        super.onDraw(canvas);
        canvas.drawColor(Color.WHITE);                  // background

        paint.setColor(Color.RED);
        paint.setStyle(Paint.Style.FILL);
        canvas.drawCircle(200, 200, 100, paint);        // cx, cy, radius

        paint.setColor(Color.BLUE);
        canvas.drawRect(100, 350, 400, 500, paint);     // left, top, right, bottom

        paint.setColor(Color.GREEN);
        paint.setStrokeWidth(8);
        canvas.drawLine(100, 600, 500, 650, paint);     // x1, y1, x2, y2

        paint.setColor(Color.BLACK);
        paint.setTextSize(48);
        canvas.drawText("Hello Canvas", 100, 800, paint);
    }
}

// In the activity:   setContentView(new DrawView(this));
// Or in XML:         <com.example.myapp.DrawView android:layout_width="match_parent"
//                        android:layout_height="match_parent" />
```

- **onDraw()** is called by the system whenever the view needs to be drawn; call `invalidate()` to ask for a redraw.
- Create the **Paint** once (as a field), not inside onDraw(), because onDraw() runs very often.
- The two constructors are needed so the view can be created in code and inflated from XML.

Topic 5: Working with Animation

Real-life example

Animation is like a **flip-book**: many slightly different pictures shown quickly look like movement. Android can also do the work of the **in-between pictures** for you.

Definition

Animation is the process of creating motion or change in a UI element over time, such as fading, moving, rotating or zooming a view.

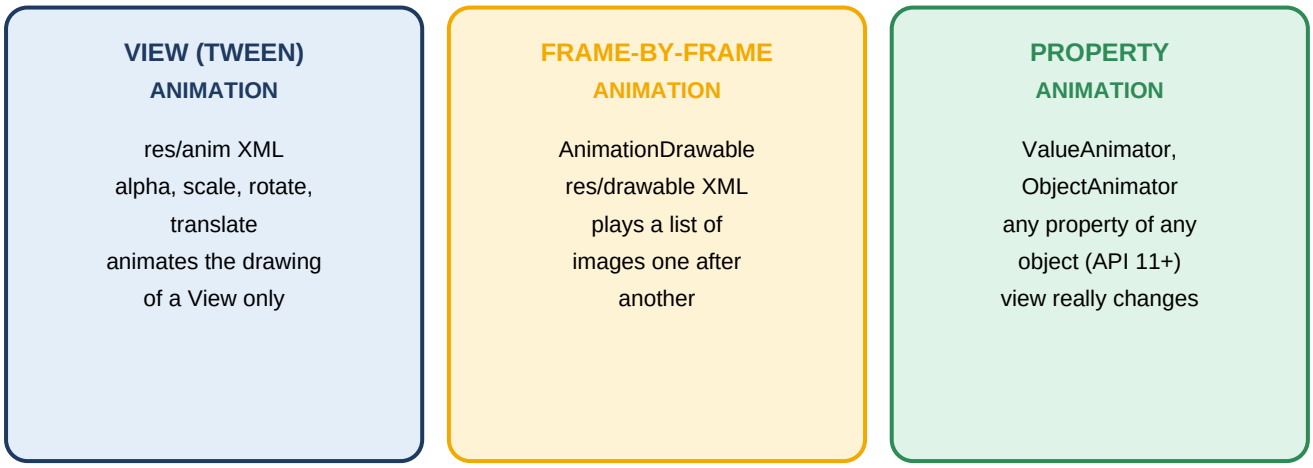


Figure 7: Types of animation in Android

Point	View (tween) animation	Frame-by-frame	Property animation
Idea	Transforms how a view is drawn	Shows a series of images	Changes real values of an object
Classes	Animation, AnimationUtils	AnimationDrawable	ObjectAnimator, ValueAnimator, AnimatorSet
Defined in	res/anim (XML) or code	res/drawable (animation-list)	Code (or res/animator)
Since	API 1	API 1	API 11

5(a) View (tween) animation

Tween means 'in between'. You give the **start and end values** and the system creates all the frames in between. There are four basic transformations.

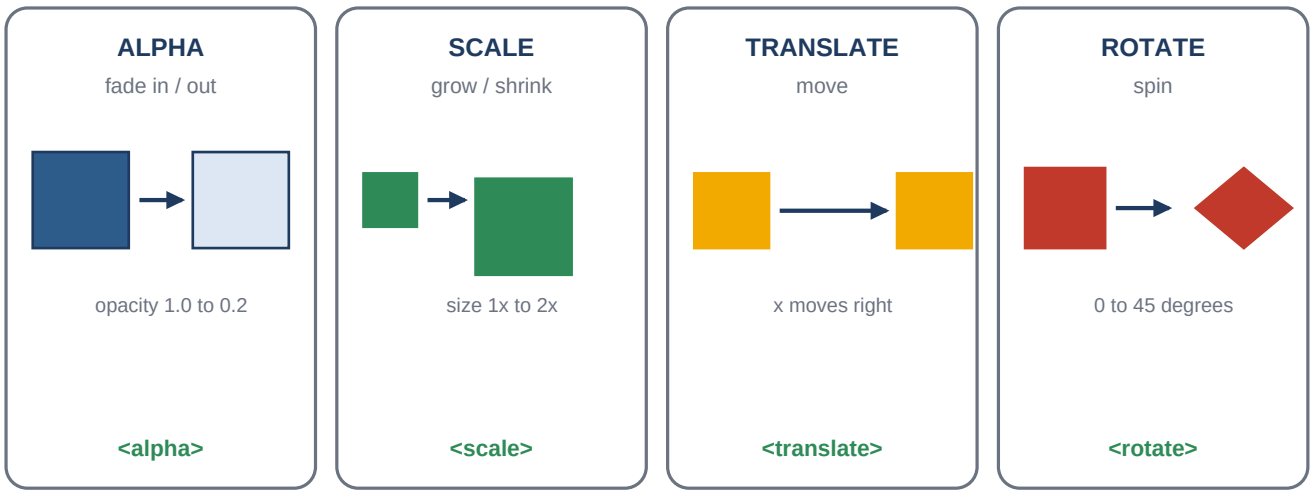


Figure 8: The four tween animations

XML tag	Effect	Main attributes
<alpha>	Fade in / fade out (transparency)	fromAlpha, toAlpha (0.0 to 1.0)
<scale>	Zoom in / zoom out	fromXScale, toXScale, fromYScale, toYScale, pivotX, pivotY
<translate>	Move the view	fromXDelta, toXDelta, fromYDelta, toYDelta
<rotate>	Rotate around a pivot	fromDegrees, toDegrees, pivotX, pivotY
<set>	Combine several animations	shareInterpolator, and children

- **Common attributes for all: duration** (ms), **startOffset** (delay), **repeatCount** (-1 or 'infinite' = forever), **repeatMode** (restart or reverse), **fillAfter** (keep the end state), **interpolator** (speed curve).

Tween animation XML files (res/anim)

```
<!-- res/anim/fade_in.xml -->
<alpha xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromAlpha="0.0" android:toAlpha="1.0" android:duration="1000" />

<!-- res/anim/rotate.xml -->
<rotate xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromDegrees="0" android:toDegrees="360"
    android:pivotX="50%" android:pivotY="50%" android:duration="1000" />

<!-- res/anim/zoom.xml -->
<scale xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromXScale="1.0" android:toXScale="2.0"
    android:fromYScale="1.0" android:toYScale="2.0"
    android:pivotX="50%" android:pivotY="50%" android:duration="1000" />

<!-- res/anim/move.xml -->
<translate xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromXDelta="0" android:toXDelta="300" android:duration="1000" />

<!-- res/anim/combo.xml: several together -->
<set xmlns:android="http://schemas.android.com/apk/res/android"
    android:interpolator="@android:anim/accelerate_decelerate_interpolator"
    android:fillAfter="true">
    <scale android:fromXScale="1.0" android:toXScale="1.5"
        android:fromYScale="1.0" android:toYScale="1.5"
        android:pivotX="50%" android:pivotY="50%" android:duration="800" />
    <alpha android:fromAlpha="1.0" android:toAlpha="0.4" android:duration="800" />
</set>
```

Java: load and start a tween animation

```
ImageView img = findViewById(R.id.imageView);
Button btn = findViewById(R.id.btnFade);

btn.setOnClickListener(v -> {
    Animation anim = AnimationUtils.loadAnimation(this, R.anim.fade_in);
    img.startAnimation(anim);
});

// Listen to the animation
anim.setAnimationListener(new Animation.AnimationListener() {
    @Override public void onAnimationStart(Animation a) { }
    @Override public void onAnimationEnd(Animation a) { /* done */ }
    @Override public void onAnimationRepeat(Animation a) { }
});

// Same thing in code, without XML
AlphaAnimation fade = new AlphaAnimation(0f, 1f);
fade.setDuration(1000);
img.startAnimation(fade);
```

Interpolator	Speed pattern
LinearInterpolator	Constant speed
AccelerateInterpolator	Starts slow, ends fast
DecelerateInterpolator	Starts fast, ends slow
AccelerateDecelerateInterpolator	Slow, fast, then slow (default feel)
BounceInterpolator	Bounces at the end
OvershootInterpolator	Goes past the end, then comes back

5(b) Frame-by-frame animation

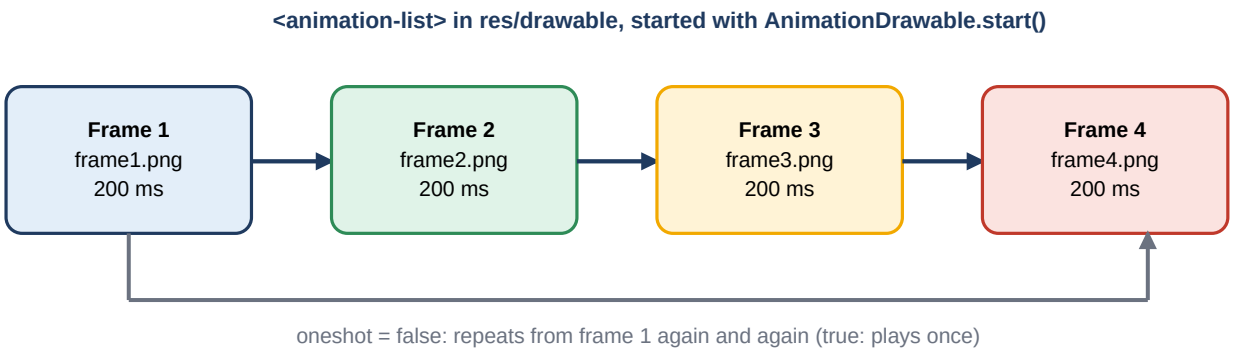


Figure 9: Frame-by-frame animation

- Several images are shown one after another, each for a set time. Defined in an **animation-list** XML file in **res/drawable** and played with **AnimationDrawable**.
- **android:oneshot**: true = play once, false = repeat forever. Each **item** has a drawable and a duration in ms.

Frame animation XML

```
<!-- res/drawable/frame_anim.xml -->
<animation-list xmlns:android="http://schemas.android.com/apk/res/android"
    android:oneshot="false">
    <item android:drawable="@drawable/frame1" android:duration="200" />
    <item android:drawable="@drawable/frame2" android:duration="200" />
    <item android:drawable="@drawable/frame3" android:duration="200" />
</animation-list>
```

Java: play the frame animation

```
ImageView img = findViewById(R.id.imgAnim);
img.setBackgroundResource(R.drawable.frame_anim);
AnimationDrawable frameAnim = (AnimationDrawable) img.getBackground();

btnStart.setOnClickListener(v -> frameAnim.start());   // start the animation
btnStop.setOnClickListener(v -> frameAnim.stop());     // stop it
```

Common mistake

Do **not** call `frameAnim.start()` inside `onCreate()`: the drawable is not yet attached to the window, so it does not play. Start it from a button click or from `onWindowFocusChanged(true)`.

5(c) Property animation

- Introduced in **API 11**. It changes the **actual property** of an object (translationX, rotation, alpha, scaleX), so the view really moves and its click area moves with it.
- **ObjectAnimator** animates a named property; **ValueAnimator** animates a value and you use it yourself; **AnimatorSet** plays animators together or in sequence.

Property animation examples

```
// Move the view 300 pixels to the right in 1 second
ObjectAnimator moveX = ObjectAnimator.ofFloat(view, "translationX", 0f, 300f);
moveX.setDuration(1000);
moveX.start();

// Rotate and fade together
ObjectAnimator rotate = ObjectAnimator.ofFloat(view, "rotation", 0f, 360f);
ObjectAnimator fade   = ObjectAnimator.ofFloat(view, "alpha", 1f, 0.2f);
AnimatorSet set = new AnimatorSet();
set.playTogether(rotate, fade);          // or set.playSequentially(rotate, fade);
set.setDuration(1500);
set.start();

// Short form with ViewPropertyAnimator
view.animate().translationX(300f).alpha(0.5f).setDuration(1000).start();

// ValueAnimator: count a progress bar from 0 to 100
ValueAnimator va = ValueAnimator.ofInt(0, 100);
va.setDuration(2000);
va.addUpdateListener(a -> progressBar.setProgress((int) a.getAnimatedValue()));
va.start();
```

Point	View (tween) animation	Property animation
What changes	Only the way the view is drawn	The real property of the object
Click area after move	Stays at the original place	Moves with the view
Works on	Views only	Any object with a property
Since	API 1	API 11

- **Screen transitions:** in older code you will see `overridePendingTransition(R.anim.fade_in, R.anim.fade_out)` after startActivity(); newer Android versions provide **overrideActivityTransition()** for the same job.
- **Tips:** keep animations short (200 to 500 ms), do not overuse them, and respect users who turn animations off in settings.

PART B: Important Questions with Model Answers (English)

Priority: **Very Likely** = appears most often in BTEUP. **Likely** = good chance. **Possible** = can appear. Each answer follows the exam format: **Definition, Points, Diagram/Table/Code, Conclusion**.

Q1. What are View and ViewGroup? Explain the UI hierarchy and the ways to create a UI. LIKELY

Definition / Introduction: A View is the basic building block of the user interface; it draws something on the screen and handles user events. A ViewGroup is an invisible container that holds other views and arranges them.

Points

- **Examples:** View: TextView, Button, ImageView. ViewGroup: LinearLayout, RelativeLayout, ConstraintLayout.
- **UI hierarchy:** the screen is a tree. The root is a ViewGroup, which contains views and other ViewGroups.
- **Ways to create a UI:** (1) XML layout files, (2) the Layout Editor, (3) Java/Kotlin code, (4) Jetpack Compose.
- **Connecting:** setContentView(R.layout.activity_main) shows the layout; findViewById(R.id.name) gets a view; setOnClickListener() handles events.
- **Units:** dp for sizes, sp for text, avoid px.

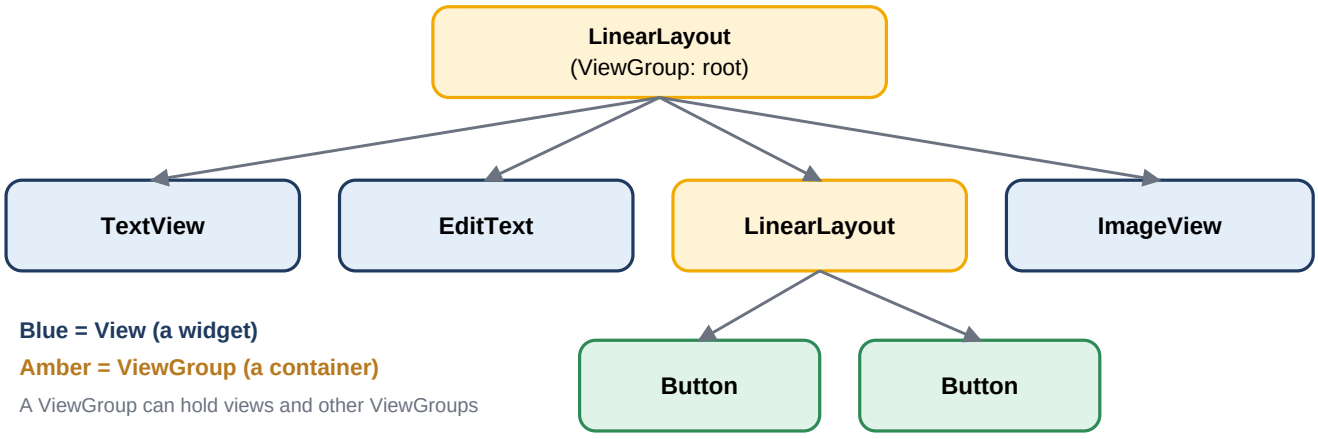


Figure: View hierarchy

Conclusion: Understanding the view hierarchy is the base of every Android UI design.

Q2. Explain TextView and EditText with their important attributes and an example.

VERY LIKELY

Definition / Introduction: TextView displays text to the user and does not allow editing. EditText is a subclass of TextView that lets the user type and edit text.

Points

- **TextView attributes:** text, textSize (sp), textColor, textStyle (bold, italic), fontFamily, gravity, maxLines.
- **EditText attributes:** hint (help text), inputType (text, textPassword, number, phone, textEmailAddress), maxLength, lines.
- The user's input is read with `getText().toString()`.
- **AutoCompleteTextView** is an EditText that shows suggestions while typing.

TextView and EditText

```
<TextView android:id="@+id/tvTitle" android:layout_width="wrap_content"
    android:layout_height="wrap_content" android:text="Login"
    android:textSize="24sp" android:textStyle="bold" />

<EditText android:id="@+id/etName" android:layout_width="match_parent"
    android:layout_height="wrap_content" android:hint="Enter your name"
    android:inputType="textPersonName" />

// Java
EditText etName = findViewById(R.id.etName);
String name = etName.getText().toString();
Toast.makeText(this, "Hello " + name, Toast.LENGTH_SHORT).show();
```

Conclusion: TextView and EditText are the most used widgets for showing and collecting text.

Q3. Explain Button, CheckBox, RadioButton and Switch with an example program.

VERY LIKELY

Definition / Introduction: These are the widgets that let users perform actions and make choices.

Points

- **Button:** performs an action when tapped; handled with `setOnClickListener()`. **ImageButton** shows an image.
- **CheckBox:** a two-state box; several can be selected. Use `isChecked()` or `setOnCheckedChangeListener()`.
- **RadioButton:** placed inside a **RadioGroup**; only one choice of the group can be selected.
- **ToggleButton / Switch:** an ON/OFF control with `setOnCheckedChangeListener()`.

Handling Button, CheckBox and RadioGroup

```
CheckBox cb = findViewById(R.id.cbAgree);
cb.setOnCheckedChangeListener((buttonView, isChecked) ->
        Toast.makeText(this, isChecked ? "Agreed" : "Not agreed",
                       Toast.LENGTH_SHORT).show());

RadioGroup rg = findViewById(R.id.rgGender);
rg.setOnCheckedChangeListener((group, checkedId) -> {
    RadioButton rb = findViewById(checkedId);
    Toast.makeText(this, rb.getText(), Toast.LENGTH_SHORT).show();
});

Button btn = findViewById(R.id.btnSubmit);
btn.setOnClickListener(v -> Toast.makeText(this, "Submitted",
                                           Toast.LENGTH_SHORT).show());
```

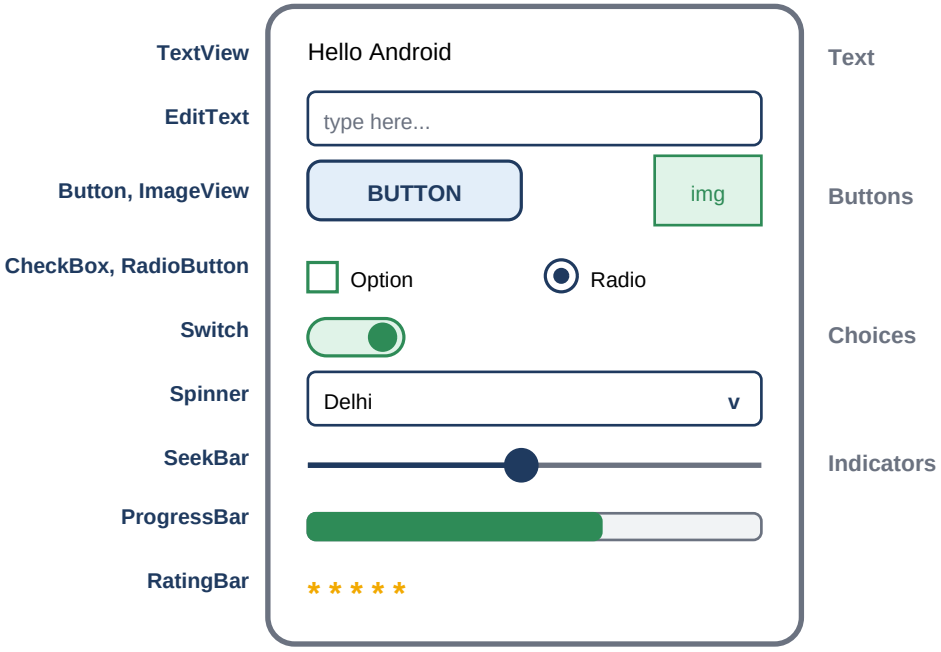


Figure: Common widgets

Conclusion: Each widget has a listener that reports the user's action to the program.

Q4. Explain Spinner and ListView with the help of Adapters. Write a program.

VERY LIKELY

Definition / Introduction: Spinner and ListView are adapter views: they show a list of items taken from a data source through an Adapter. A Spinner is a drop-down list and a ListView is a scrolling list.

Points

- **Adapter:** connects the data (array, list, database) to the view and creates a row for each item. Types: ArrayAdapter, SimpleAdapter, CursorAdapter.
- **Spinner:** user selects one item; use `setOnItemSelectedListener()`.
- **ListView:** user taps an item; use `setOnItemClickListener()`.
- **RecyclerView** is the modern replacement for large or complex lists.

ListView and Spinner with ArrayAdapter

```
String[] items = {"Android", "Java", "Kotlin", "XML"};
ListView lv = findViewById(R.id.listView);
ArrayAdapter<String> ad = new ArrayAdapter<>(
        this, android.R.layout.simple_list_item_1, items);
lv.setAdapter(ad);
lv.setOnItemClickListener((parent, view, position, id) ->
        Toast.makeText(this, items[position], Toast.LENGTH_SHORT).show());

Spinner sp = findViewById(R.id.spCity);
ArrayAdapter<CharSequence> adapter = ArrayAdapter.createFromResource(
        this, R.array.cities, android.R.layout.simple_spinner_item);
adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
sp.setAdapter(adapter);
```

Conclusion: Adapters separate the data from its display, so the same view can show any kind of data.

Q5. Write short notes on ProgressBar, SeekBar, RatingBar, DatePicker and TimePicker.

LIKELY

Definition / Introduction: These are indicator and picker widgets used to show progress and to take values such as ratings, dates and times.

Points

- **ProgressBar:** shows progress or loading; circular (indeterminate) or horizontal with `max` and `setProgress()`.
- **SeekBar:** a slider the user drags; `OnSeekBarChangeListener` gives the value in onProgressChanged().
- **RatingBar:** star rating; attributes numStars and stepSize; `getRating()` returns the rating.
- **DatePicker / TimePicker:** choose a date or time; **DatePickerDialog** and **TimePickerDialog** show them as dialogs. In the date callback, **month starts at 0**.
- **TextClock** shows the time; AnalogClock and DigitalClock are deprecated.

SeekBar and RatingBar

```
SeekBar seek = findViewById(R.id.seekBar);
seek.setOnSeekBarChangeListener(new SeekBar.OnSeekBarChangeListener() {
    @Override public void onProgressChanged(SeekBar bar, int progress, boolean fromUser) {
        textView.setText("Value: " + progress);
    }
    @Override public void onStartTrackingTouch(SeekBar bar) { }
    @Override public void onStopTrackingTouch(SeekBar bar) { }
});
RatingBar rb = findViewById(R.id.ratingBar);
float stars = rb.getRating();
```

Conclusion: These widgets make data entry and feedback easy and visual.

Q6. What is a layout? Explain the types of layouts in Android. VERY LIKELY

Definition / Introduction: A layout is a ViewGroup that defines the structure and arrangement of the views on a screen. Layouts are defined in XML files in res/layout or created in code.

Points

- **LinearLayout:** children in one row or column (orientation); supports weights.
- **RelativeLayout:** each child positioned relative to the parent or a sibling.
- **FrameLayout:** children stacked on top of each other.
- **TableLayout:** children in rows and columns using TableRow. **GridLayout:** a grid with rows and columns.
- **ConstraintLayout:** views positioned with constraints; flat hierarchy; default in templates.
- **Common attributes:** layout_width and layout_height (match_parent, wrap_content), padding, layout_margin, gravity. AbsoluteLayout is deprecated.

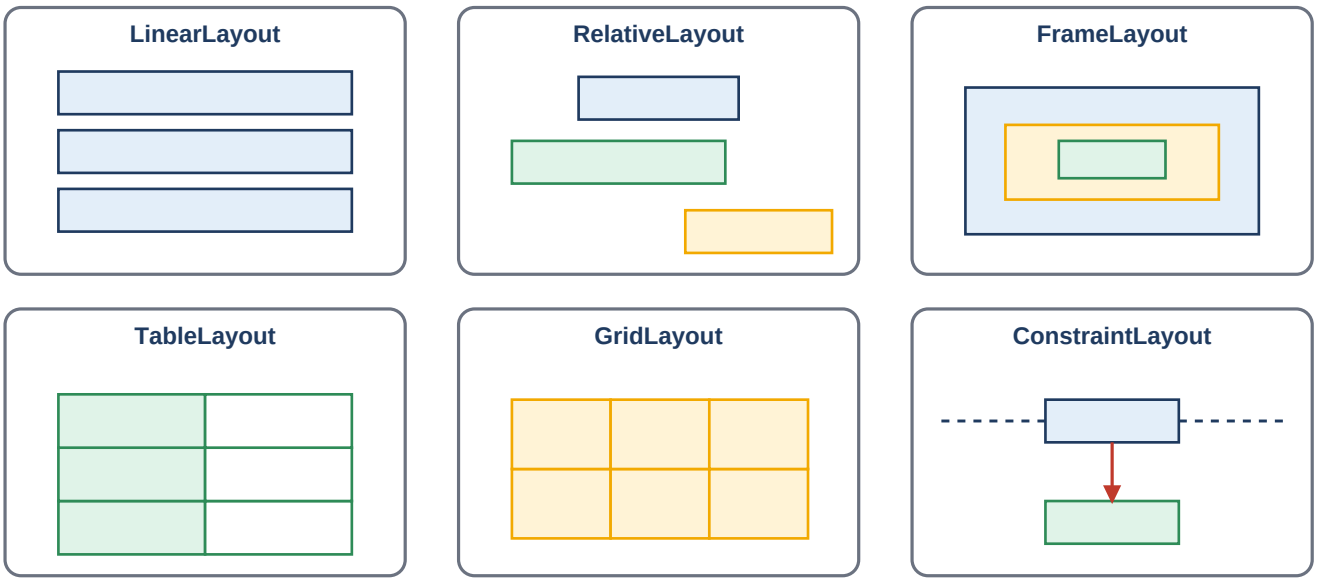


Figure: Types of layouts

Conclusion: Choosing the right layout gives a clean, flexible and fast user interface.

Q7. Explain LinearLayout with its attributes and write an XML example. VERY LIKELY

Definition / Introduction: LinearLayout arranges all its children in a single direction, either vertically in a column or horizontally in a row.

Points

- **android:orientation:** vertical (top to bottom) or horizontal (left to right, default).
- **android:layout_weight:** shares the remaining space among the children in a ratio; set the size to 0dp when using it.
- **android:gravity:** position of the content inside a view. **android:layout_gravity:** position of the view inside its parent.
- **padding / layout_margin:** space inside / outside a view.
- Layouts can be nested, for example horizontal LinearLayouts inside a vertical one.

LinearLayout: nested vertical and horizontal

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:orientation="vertical" android:padding="16dp">

    <TextView android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:text="Register" android:textSize="22sp" />

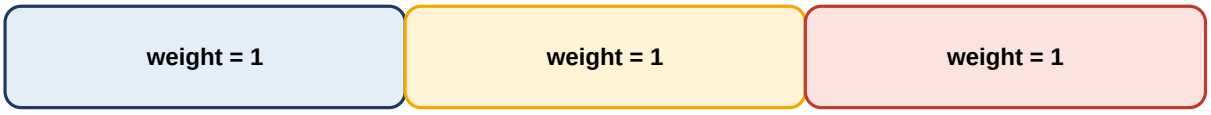
    <EditText android:layout_width="match_parent" android:layout_height="wrap_content"
        android:hint="Name" />

    <LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
        android:orientation="horizontal">
        <Button android:layout_width="0dp" android:layout_height="wrap_content"
            android:layout_weight="1" android:text="Save" />
        <Button android:layout_width="0dp" android:layout_height="wrap_content"
            android:layout_weight="1" android:text="Cancel" />
    </LinearLayout>
</LinearLayout>
```

layout_weight 1 and 2 (width = 0dp)



three views with equal weight 1 : 1 : 1



Free space is shared in the ratio of the weights.

Figure: layout_weight

Conclusion: LinearLayout is the simplest layout for forms and rows of buttons.

Q8. Explain RelativeLayout with its important attributes and an example. VERY LIKELY

Definition / Introduction: RelativeLayout positions each child relative to the parent layout or to other sibling views, which gives flexible designs without deep nesting.

Points

- **Relative to the parent:** layout_alignParentTop / Bottom / Start / End, layout_centerInParent, layout_centerHorizontal, layout_centerVertical.
- **Relative to a sibling (by id):** layout_above, layout_below, layout_toLeftOf, layout_toRightOf (or Start/End), layout_alignLeft, layout_alignTop, layout_alignBaseline.
- Ids are referenced as `@id/name`; the referenced view should be declared first.

RelativeLayout example

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:padding="16dp">

    <TextView android:id="@+id/tvName" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:layout_centerHorizontal="true"
        android:text="Name" />

    <EditText android:id="@+id/etName" android:layout_width="match_parent"
        android:layout_height="wrap_content" android:layout_below="@id/tvName" />

    <Button android:id="@+id/btnOk" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:layout_below="@id/etName"
        android:layout_alignParentEnd="true" android:text="OK" />
</RelativeLayout>
```

Conclusion: RelativeLayout is useful when the position of one view depends on another.

Q9. Write short notes on FrameLayout, TableLayout, GridLayout and ConstraintLayout. LIKELY

Definition / Introduction: Besides LinearLayout and RelativeLayout, Android provides these layouts for special needs.

Points

- **FrameLayout:** stacks children on top of each other; the last child is on top; use layout_gravity. Used for overlays and for fragments.
- **TableLayout:** arranges children in rows (TableRow) and columns; **stretchColumns** and **shrinkColumns** control the width.
- **GridLayout:** a grid defined by columnCount and rowCount; children can span cells.
- **ConstraintLayout:** each view has horizontal and vertical constraints to the parent or siblings; flat hierarchy; good for complex, responsive screens.

TableLayout example

```
<TableLayout android:layout_width="match_parent" android:layout_height="wrap_content"
    android:stretchColumns="1">
    <TableRow>
        <TextView android:text="Name:" />
        <EditText android:hint="Enter name" />
    </TableRow>
    <TableRow>
        <TextView android:text="Email:" />
        <EditText android:hint="Enter email" />
    </TableRow>
</TableLayout>
```

Layout	Arrangement	Best for
FrameLayout	Stack	Overlays
TableLayout	Rows, columns	Forms
GridLayout	Grid	Keypads
ConstraintLayout	Constraints	Complex screens

Conclusion: Using the right layout for each screen keeps the code simple and the UI efficient.

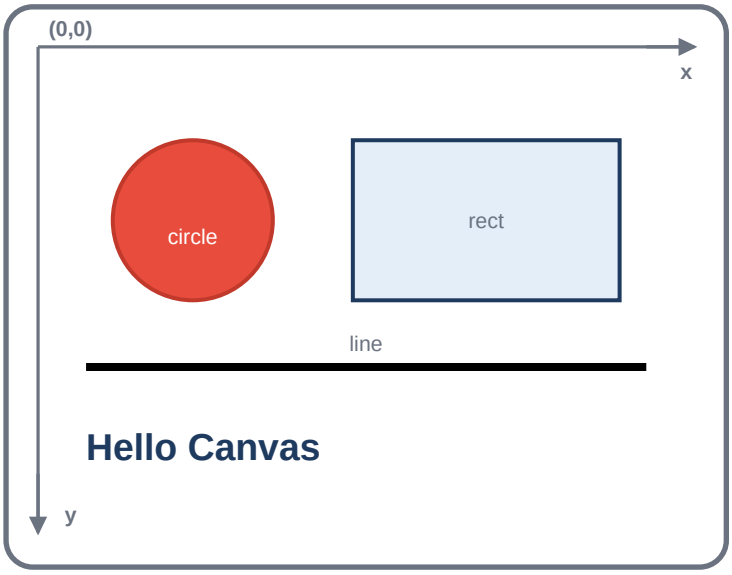
Q10. Explain drawing in Android: Canvas, Paint, shapes, text and bitmaps. Write a custom View program.

VERY LIKELY

Definition / Introduction: Drawing in Android means creating graphics on the screen. It needs a Canvas (the drawing surface), a Paint (how to draw) and the thing to draw (shape, text or bitmap).

Points

- **Canvas:** has methods drawCircle(), drawRect(), drawLine(), drawText(), drawBitmap(). The origin (0, 0) is the top-left corner.
- **Paint:** colour, style (FILL or STROKE), stroke width, text size, anti-aliasing.
- **Shapes:** drawn in code, or defined as an XML shape drawable (rectangle, oval, line, ring) with solid, stroke, corners and gradient.
- **Text:** canvas.drawText() with Paint.setTextSize(); TextView for normal text.
- **Bitmaps:** BitmapFactory.decodeResource() loads an image; ImageView or canvas.drawBitmap() shows it; scale big images to avoid OutOfMemoryError.



CANVAS draws, PAINT styles

- drawCircle(cx, cy, radius, paint)
- drawRect(left, top, right, bottom, paint)
- drawLine(x1, y1, x2, y2, paint)
- drawText(text, x, y, paint)
- drawBitmap(bitmap, x, y, paint)
- drawColor(color)

Figure: Canvas and drawing methods

Custom View with onDraw()

```
public class DrawView extends View {
    private final Paint paint = new Paint(Paint.ANTI_ALIAS_FLAG);

    public DrawView(Context context) { super(context); }

    @Override
    protected void onDraw(Canvas canvas) {
        super.onDraw(canvas);
        paint.setColor(Color.RED);
        canvas.drawCircle(200, 200, 100, paint);
        paint.setColor(Color.BLUE);
        canvas.drawRect(100, 350, 400, 500, paint);
        paint.setColor(Color.BLACK);
        paint.setTextSize(48);
        canvas.drawText("Hello Canvas", 100, 800, paint);
    }
}
// Activity: setContentView(new DrawView(this));
```

Conclusion: Canvas and Paint allow complete control over what appears on the screen.

Q11. What is animation? Explain the types of animation in Android. VERY LIKELY

Definition / Introduction: Animation is the process of creating motion or change in a UI element over time, such as fading, moving, rotating or zooming a view. It makes an app attractive and easier to understand.

Points

- **View (tween) animation:** alpha, scale, translate and rotate, defined in XML (res/anim) or code; it changes only how the view is drawn.
- **Frame-by-frame animation:** a series of images played one after another using an animation-list and AnimationDrawable.
- **Property animation (API 11):** changes real property values using ObjectAnimator, ValueAnimator and AnimatorSet; works on any object.
- **Interpolators** control the speed: linear, accelerate, decelerate, bounce, overshoot.

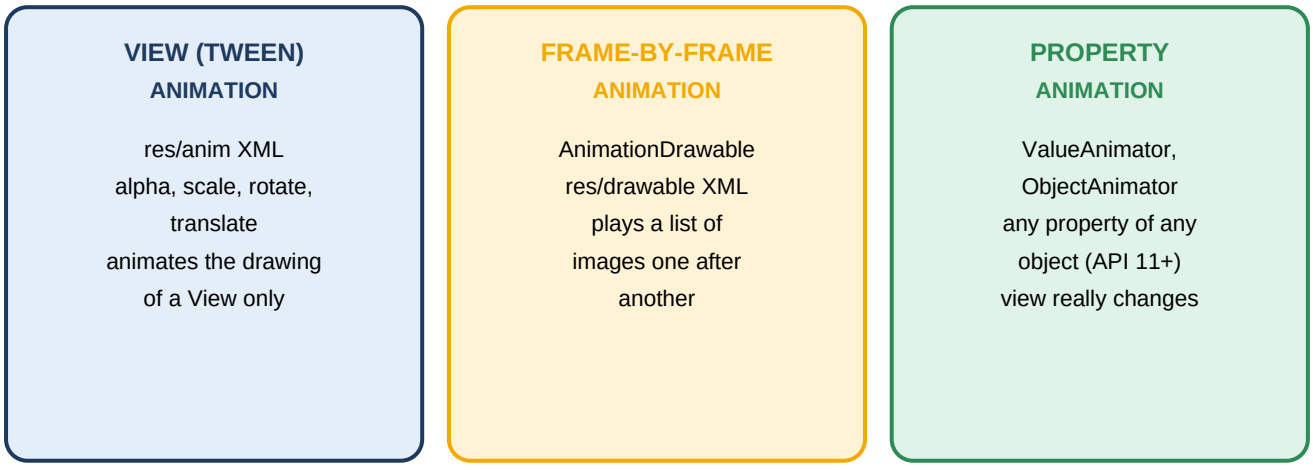


Figure: Types of animation

Conclusion: Android offers a simple tool for each kind of animation, from a quick fade to a complex object movement.

Q12. Explain tween animation. Write XML and Java code for alpha, scale, translate and rotate animations.

VERY LIKELY

Definition / Introduction: Tween animation applies simple transformations to a view. The developer gives the start and end values and the system calculates the frames in between.

Points

- **Alpha:** changes transparency (fade in/out); fromAlpha and toAlpha.
- **Scale:** zooms in/out; fromXScale, toXScale, fromYScale, toYScale, pivotX, pivotY.
- **Translate:** moves the view; fromXDelta, toXDelta, fromYDelta, toYDelta.
- **Rotate:** turns the view; fromDegrees, toDegrees, pivotX, pivotY.
- **Common:** duration, startOffset, repeatCount, repeatMode, fillAfter, interpolator. Use **<set>** to combine animations.
- **Steps:** create the XML in res/anim, load it with `AnimationUtils.loadAnimation()`, start it with `view.startAnimation()`.

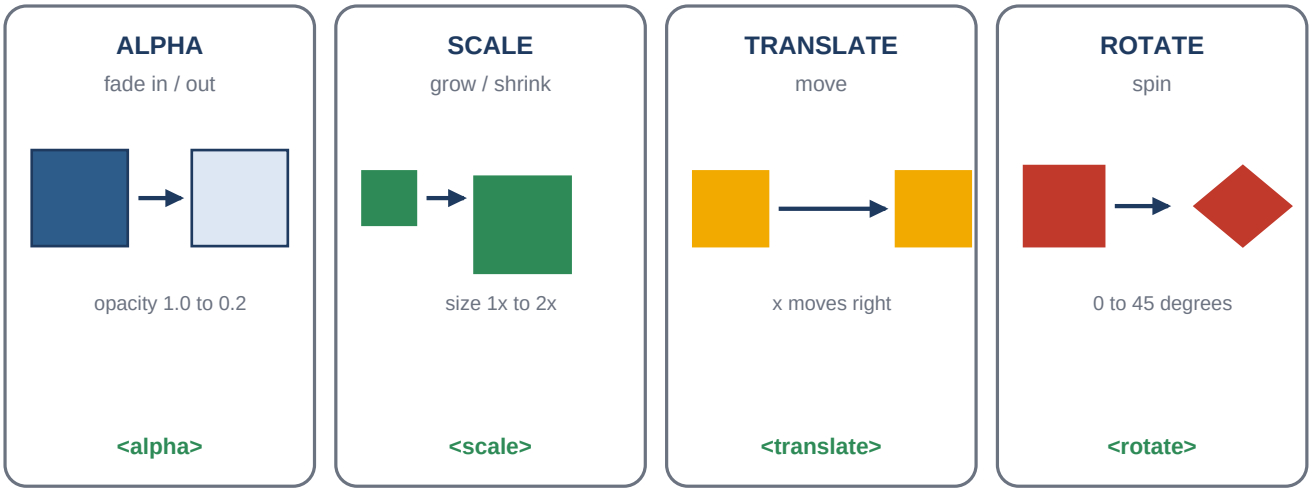


Figure: The four tween animations

Tween animations in XML

```
<!-- res/anim/fade_in.xml -->
<alpha xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromAlpha="0.0" android:toAlpha="1.0" android:duration="1000" />

<!-- res/anim/zoom.xml -->
<scale xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromXScale="1.0" android:toXScale="2.0"
    android:fromYScale="1.0" android:toYScale="2.0"
    android:pivotX="50%" android:pivotY="50%" android:duration="1000" />

<!-- res/anim/move.xml -->
<translate xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromXDelta="0" android:toXDelta="300" android:duration="1000" />

<!-- res/anim/rotate.xml -->
<rotate xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromDegrees="0" android:toDegrees="360"
    android:pivotX="50%" android:pivotY="50%" android:duration="1000" />
```

Java: start the animation

```
ImageView img = findViewById(R.id.imageView);
Animation anim = AnimationUtils.loadAnimation(this, R.anim.fade_in);
img.startAnimation(anim);
```

Conclusion: Tween animations are the quickest way to add fade, zoom, move and rotate effects.

Q13. Explain frame-by-frame animation with an example. LIKELY

Definition / Introduction: Frame-by-frame animation shows a sequence of images one after another, each for a fixed time, like a flip-book. It is defined with an animation-list and played with AnimationDrawable.

Points

- Put the frame images (frame1.png, frame2.png, ...) in **res/drawable**.
- Create an **animation-list** XML with an **item** for each frame, with its drawable and duration (ms). **oneshot** = false repeats the animation.
- Set it as the background of an ImageView, get the **AnimationDrawable** and call **start()** or **stop()**.
- Do not start it in onCreate(); start it from a button click or onWindowFocusChanged().

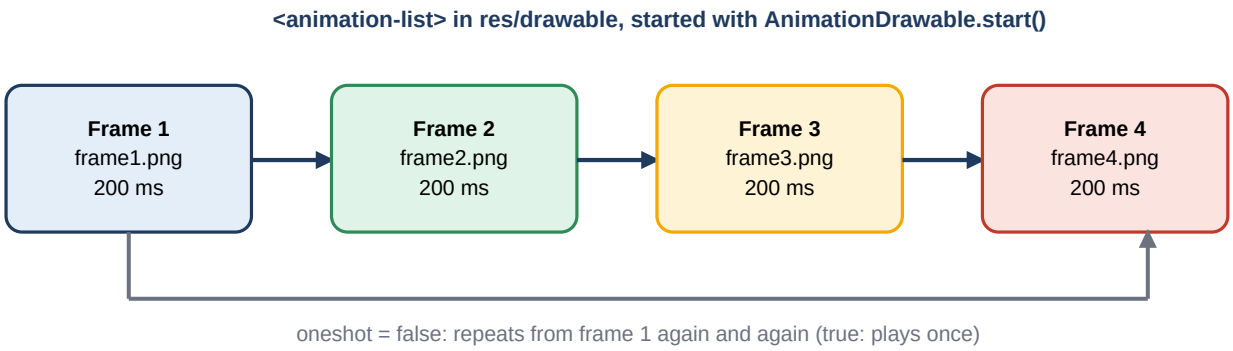


Figure: Frame-by-frame animation

Frame animation

```
<!-- res/drawable/frame_anim.xml -->
<animation-list xmlns:android="http://schemas.android.com/apk/res/android"
    android:oneshot="false">
    <item android:drawable="@drawable/frame1" android:duration="200" />
    <item android:drawable="@drawable/frame2" android:duration="200" />
    <item android:drawable="@drawable/frame3" android:duration="200" />
</animation-list>

// Java
ImageView img = findViewById(R.id.imgAnim);
img.setBackgroundResource(R.drawable.frame_anim);
AnimationDrawable frameAnim = (AnimationDrawable) img.getBackground();
btnStart.setOnClickListener(v -> frameAnim.start());
```

Conclusion: Frame animation is best for short, hand-drawn style animations such as a loading icon.

Q14. Differentiate between: (a) View animation and Property animation, (b) dp and sp, (c) match_parent and wrap_content, (d) LinearLayout and RelativeLayout.

POSSIBLE

Definition / Introduction: These pairs are often confused. The tables compare them.

Points

- Each table compares the two items point by point.

(a) Point	View animation	Property animation
Changes	Only how the view is drawn	Real property of the object
Click area	Stays at old place	Moves with the view
Works on	Views only	Any object
Since	API 1	API 11

(b) Point	dp	sp
Full form	Density-independent pixel	Scale-independent pixel
Used for	Sizes, margins, padding	Text size
Follows font setting	No	Yes

(c) Point	match_parent	wrap_content
Size	As big as the parent	Just big enough for the content

(d) Point	LinearLayout	RelativeLayout
Arrangement	One row or column	Relative to parent or siblings
Key attribute	orientation, layout_weight	layout_below, layout_toRightOf, alignParent...
Best for	Simple forms	Position depends on other views

Conclusion: Knowing these differences helps choose the right tool for each UI task.

Quick Revision Sheet (read the night before)

Term	Remember this
View / ViewGroup	Widget / container of views (layout)
Units	dp for size, sp for text, avoid px
layout_width values	match_parent, wrap_content, or dp
visibility	visible, invisible (keeps space), gone (no space)
Event handling	findViewById + setOnClickListener
EditText	inputType, hint, getText().toString()
RadioButton	Inside RadioGroup: only one selected
Spinner / ListView	Adapter views; ArrayAdapter; OnItemSelected / OnItemClick
Date picker month	Starts at 0 (January = 0)
Deprecated	AnalogClock, DigitalClock, Gallery, ProgressDialog, AbsoluteLayout
LinearLayout	orientation, layout_weight (0dp), gravity vs layout_gravity
RelativeLayout	layout_below, layout_toRightOf, layout_alignParent..., layout_centerInParent
Frame / Table / Grid	Stack / rows (TableRow) / columnCount
ConstraintLayout	Constraints to parent or siblings; flat; 0dp = match constraints
Canvas / Paint	Surface + methods / colour, style, size. Origin top-left
Shape drawable	shape = rectangle, oval, line, ring; solid, stroke, corners, gradient
Bitmap	BitmapFactory.decodeResource; scale; OutOfMemoryError risk
Tween animation	alpha, scale, translate, rotate (+ set); res/anim; AnimationUtils.loadAnimation
Frame animation	animation-list + AnimationDrawable; start() not in onCreate
Property animation	ObjectAnimator, ValueAnimator, AnimatorSet; API 11

Memory tricks

Tween animations: A-S-T-R = Alpha, Scale, Translate, Rotate.
Layouts: L-R-F-T-G-C = Linear, Relative, Frame, Table, Grid, Constraint.
gravity = **inside** the view. **layout_gravity** = the view **in its parent**.
dp = **d**istance, **sp** = **s**cale of text.
Animation types: V-F-P = View (tween), Frame, Property.

Common Mistakes Students Make

- Using **px** instead of **dp** and **sp**. The UI then looks different on every phone.
- Mixing up **gravity** and **layout_gravity**.
- Using weights without setting the size to **0dp**.
- Writing deprecated things (AbsoluteLayout, ProgressDialog, DigitalClock) as the normal way.
- Forgetting the **duration** in an animation, or calling frame **start()** in onCreate().
- Forgetting that the **date-picker month starts at 0**.

Self-Test (answer without looking, 5 marks each)

- Write the XML of a vertical LinearLayout with a TextView, an EditText and two buttons side by side using weights.
- Differentiate between LinearLayout and RelativeLayout. Name the other layouts.
- Write a program to show a Toast with the selected Spinner item.
- Explain tween animation. Write the XML for a fade-in and a rotate animation.
- What are Canvas and Paint? Write the code to draw a circle.

Self-Test Answers (check after writing)

1. LinearLayout orientation=vertical containing TextView, EditText (match_parent width) and an inner horizontal LinearLayout with two Buttons: layout_width=0dp and layout_weight=1 for both. See Q7.

2. LinearLayout: one row or column, orientation and weight. RelativeLayout: positions relative to parent or siblings. Others: FrameLayout, TableLayout, GridLayout, ConstraintLayout.

3. Set an ArrayAdapter on the Spinner, then setOnItemSelectedListener: in onItemSelected() use parent.getItemAtPosition(pos).toString() and Toast.makeText(...).show(). See Q4.

4. Tween animation gives start and end values and the system creates the frames between. Fade-in: <alpha fromAlpha="0.0" toAlpha="1.0" duration="1000"/>. Rotate: <rotate fromDegrees="0" toDegrees="360" pivotX="50%" pivotY="50%" duration="1000"/>.

5. Canvas is the drawing surface with draw methods; Paint holds colour, style and size. Code: Paint p = new Paint(); p.setColor(Color.RED); canvas.drawCircle(200, 200, 100, p); inside onDraw() of a custom View.

> **Done with Unit III!**
> If you scored 4 out of 5 or more, move to **Unit IV**. If not, re-read the topics you missed in Part A.